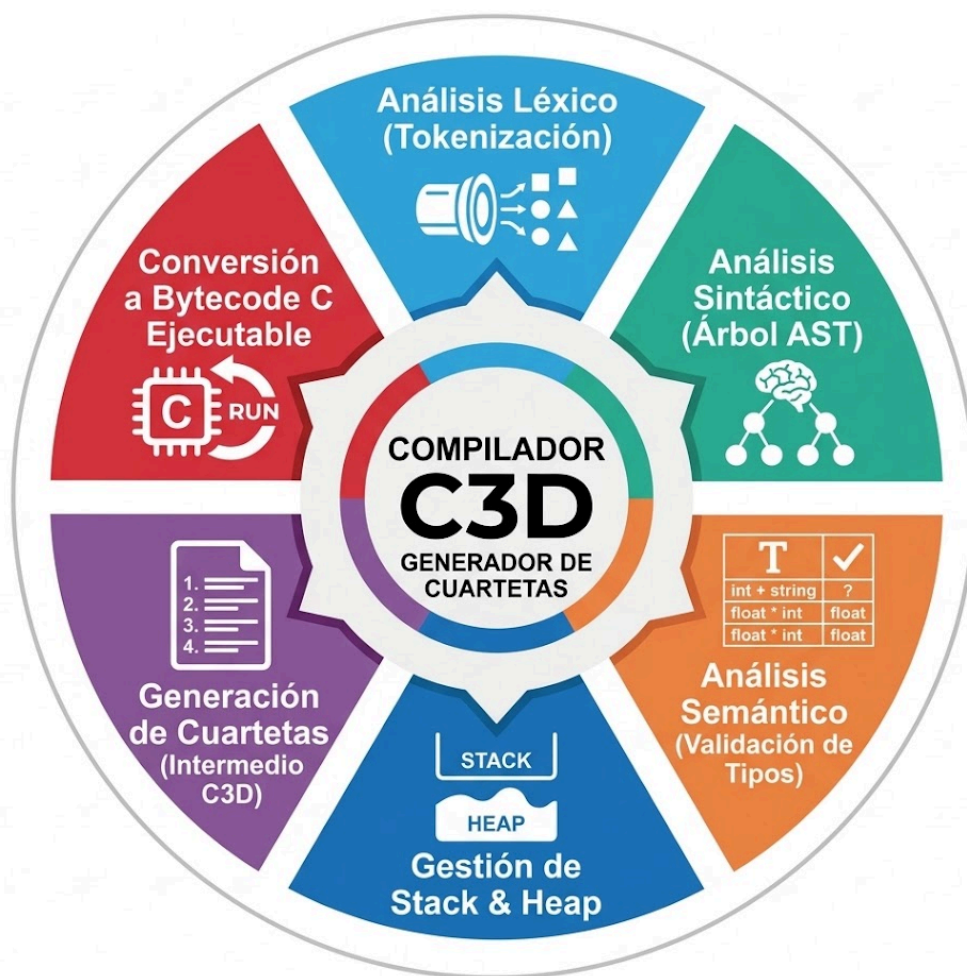


MANUAL TÉCNICO

IDE XS1



Javier Alejandro Mérida Gómez 202230638

26/09/2026

ÍNDICE

ÍNDICE	2
DESCRIPCIÓN GENERAL	3
OBJETIVOS	3
GENERAL	3
ESPECÍFICOS	3
ESPECIFICACIONES DEL SISTEMA	4
REQUISITOS DE HARDWARE Y SOFTWARE	4
TECNOLOGÍAS	4
REPOSITORIO EN GITHUB	4
ORGANIZACIÓN DEL PROYECTO	5
ANÁLISIS DE LOS REQUERIMIENTOS	5
PIGLATIN	5
Y	10
ZETARIANO	14
ANALIZADOR SEMÁNTICO	19
TABLA DE COLOREADO	20
INTERFAZ GRÁFICA	21

DESCRIPCIÓN GENERAL

IDE capaz de soportar 3 lenguajes de alto nivel, capaz de poder validarlo léxica, sintáctica y semánticamente, para luego descompenerlo en código tres direcciones o código intermedio por medio de cuartetas y generar un archivo en lenguaje C, completamente funcional .

Mostrando las tablas, árboles abstractos, cuartetas, código tres direcciones y código , obtenidas durante el análisis.

OBJETIVOS

GENERAL

- Generar código de tres direcciones a partir de lenguajes de alto nivel como parte del proceso de compilación.

ESPECÍFICOS

- Validar los lenguajes de alto nivel de manera léxica, sintáctica y semántica antes de iniciar el proceso de compilación.
- Generar las estructuras adecuadas para tener una representación de cuartetas del programa.
- Utilizar las cuartetas generadas para producir código funcional en el lenguaje de programación C.
- Poner en práctica los conceptos de stack y heap en el proceso de compilación.

ESPECIFICACIONES DEL SISTEMA

REQUISITOS DE HARDWARE Y SOFTWARE

- Sistemas Operativos Compatibles: Microsoft Windows 10 (o superior) / GNU/Linux (Ubuntu 20.04 LTS o superior).
- Entorno de Desarrollo: Visual Studio Code y Apache NetBeans.

TECNOLOGÍAS

Desarrollo	Tecnología / Librería	Función
Analizadores (léxico y sintáctico)	ANTLR4 4.13.2	Generar el lexer y el parser de cada lenguaje a partir de su gramática (pigLatin.g4 , y.g4 ,Zetariano.g4), junto con las clases base Visitor
Interfaz gráfica	Java Swing	Interfaz de escritorio: árbol del proyecto, editor con pestañas, tabla de errores, panel de código C y diálogos
Backen	Java 21	AST, tablas de símbolos y tipos, análisis semántico, cuartetos, C3D y traducción a C
Construcción	Apache Maven	Dependencias y compilación; incluye src/main/antlr4 como carpeta de código
Graficación del AST	Graphviz	Dibujar los AST como grafos navegables (zoom, desplazamiento, exportar a PNG)
Código final	C + gcc	Programa generado a partir del C3D; se compila con gcc desde el IDE

REPOSITORIO EN GITHUB

Link del repositorio:
<https://github.com/JavierMeridaLk/ProyectoUnoCompi2SS26>

ORGANIZACIÓN DEL PROYECTO

Paquete	Contenido
analizador	Análisis léxico/sintáctico (Analizador), carga del programa y sus imports (CargadorPrograma) y análisis semántico(AnalizadorSemantico , ContextoSemantico)
ast	Nodos del AST de cada lenguaje (piglatin , y , zetariano) con su interfaz Visitor , y piezas comunes (comun : Tipo ,Posicion , operadores)
visitors	Un paquete por lenguaje con sus visitors: AstBuilder , ConstructorTablas... , Arbol... y Generador...
tablas	Tabla de símbolos, tabla de tipos, tabla de compatibilidad y su construcción
cuartetas	Cuartetas, generador base, traducción a C3D y a C, compilación con gcc
errores	Manejo y reporte de errores léxicos, sintácticos y semánticos
archivos	Interfaz gráfica (coloreado , componentes)
views	Interfaz gráfica (coloreado , componentes)
com.mycompany	Lexers, parsers y visitors generados por ANTLR (carpeta src/main/antlr4)

ANÁLISIS DE LOS REQUERIMIENTOS

El compilador reconoce tres lenguajes. Cada uno tiene su propia gramática de ANTLR genera su lexer, su parser y su clase base Visitor. Los errores léxicos no detienen el análisis: los símbolos inválidos se envían a un canal oculto y se reportan; los errores sintácticos se reportan con una estrategia de recuperación en español (EstrategiaErroresSintacticos) que indica qué se esperaba y, cuando falta un ; , lo marca al final de la instrucción anterior.

PIGLATIN

Analizador sintáctico — gramática

programa

: importacion* seccionVariables? seccionPrincipal EOF

importacion

: 'import' rutaImportacion ';'?

rutaImportacion

: IDENTIFICADOR ('.' IDENTIFICADOR)+

seccionVariables

: 'VARIABLES' '>' declaracion*

dimension

: '[' expresion ']'

inicializador

: expresion

| listaInicializacion

listaInicializacion

: '{' (inicializador ('.' inicializador)*)? '}'

declaracion

: declaracionVariable

| declaracionArreglotipo

: 'numerus'

| 'textum'

| 'decimalis'

| 'littera'

| 'bool'

| IDENTIFICADOR

declaracionVariable

: 'esto' IDENTIFICADOR ':' cuerpoDeclaracion ';'bloque

```

: '{' instruccion* '}'
cuerpoDeclaracion
: tipo inicializador?
| ('verum' | 'falsus')
| creacionObjetoinstruccion
: declaracion
| asignacion
| incremento
| llamada
| condicional
| cicloDum
| cicloFacere
| cicloPer
| continuar
| interrumpir
| lectura
| escritura
actualizacionPer
: acceso ('++' | '--')
| acceso '=' expresion
lectura
: acceso? '<<' ';' ?
asignacion
: acceso '=' expresion ';' escritura
: '>>' expresion ('>>' expresion)* ';'

```

incremento

: acceso ('++' | '--') ';' acceso

: IDENTIFICADOR sufijo*

llamada

: acceso ';' sufijo

: '[' expresion ']'

| '.' IDENTIFICADOR

| '(' argumentos? ')'

continuar

: 'perge' ';'

argumentos

: expresion (',' expresion)*

interrumpir

: 'interrumpe' ';'

condicional

: 'si' '(' expresion ')' bloque sinoSi* sino? 'finis' ';'

sinoSi

: 'aliter' '(' expresion ')' bloque

sino

: 'aliter' bloque

cicloDum

: 'dum' '(' expresion ')' bloque 'finis' ';'

cicloFacere

: 'facere' bloque 'dum' '(' expresion ')' ';'

creacionObjeto

: 'novus' IDENTIFICADOR '(' argumentos? ')'

expresion

: '(' expresion ')'

| literal

| creacionObjeto sufijo*

| acceso

| ('non' | '!' | '-') expresion

| expresion ('*' | '/') expresion

| expresion ('+' | '-') expresion

| expresion ('<' | '<=' | '>' | '>=') expresion

| expresion ('==' | '!=') expresion

| expresion '&&' expresion

| expresion '||' expresion

cicloPer

literal

: 'per' '(' inicioPer ';' expresion ';' actualizacionPer ')' bloque ('finis'

: ENTERO

;)?)

| DECIMAL

inicioPer

| TEXTO

: 'esto' IDENTIFICADOR ':' tipo expresion

| CHARACTER

| acceso '=' expresion

| 'verum'

| 'falsus'

Y

programa

: NEWLINE* seccionEstructuras? seccionFunciones EOFIncremento

: acceso ('++' | '--')

seccionEstructuras

: '%estructuras' NEWLINE definicionEstructura*retornar

: 'retornar' expresion?

seccionFunciones

: '%funciones' NEWLINE definicionFuncion*imprimir

: 'imprimir' '(' argumentos? ')'

definicionEstructura

leer

: 'estructura' IDENTIFICADOR ':' NEWLINE INDENT atributoEstructura+
DEDENT

: 'leer' '(' ')'

atributoEstructura

: tipo IDENTIFICADOR dimension* NEWLINE

llamadaFuncion

: IDENTIFICADOR '(' argumentos? ')'

definicionFuncion

argumentos

: 'definir' IDENTIFICADOR '(' parametros? ') ('->' tipoRetorno)? ':' :

NEWLINE

expresion

bloque

(',' expresion)*

tipoRetorno

: tipo ('[' ']')*condicional

: 'si' '(' expresion ')' 'entonces' NEWLINE bloque sinoSi* contr

parametros

: parametro (',' parametro)*sinoSi

: 'sino' '(' expresion ')' 'entonces' NEWLINE bloque

parametro

: tipo IDENTIFICADOR

| ('[' ']')+ tipo IDENTIFICADOR

| '{' '}' IDENTIFICADOR IDENTIFICADORcontrario

: 'contrario' NEWLINE bloque

tipo

: 'entero'

| 'flotante'

| 'cadena'

| 'caracter'

| 'bool'

| IDENTIFICADOR

bloque

: INDENT instruccion+ DEDENT

instruccion

: instruccionSimple ';' NEWLINE

| instruccionCompuesta

instruccionSimple

: declaracion

| asignacion

| incremento

| llamadaFuncion

| imprimir

| leer

| 'romper'

| 'continuar'

| retornar

instruccionCompuesta

: definicionEstructura

| condicional

| elegir

| cicloPara

| cicloMientras

| cicloHacer

declaracion

: tipo IDENTIFICADOR dimension* ('=' inicializador)?

dimension

: '[' expresion ']'

inicializador

: expresion

| listaInicializacion

listaInicializacion

: '{ (inicializador (',' inicializador)*)? }'

asignacion

: acceso '=' inicializador

elegir

: 'elegir' '(' expresion ')' ':' NEWLINE INDENT caso* casoSiempr

caso

: 'caso' expresion ':' NEWLINE bloque

casoSiempre

: 'siempre' ':' NEWLINE bloque

cicloPara

: 'para' '(' inicioPara? ';' expresion? ';' actualizacionPara? '

inicioPara

: declaracion

| asignacion

actualizacionPara

: incremento

| asignacion

cicloMientras

: 'mientras' '(' expresion ')' 'hacer' NEWLINE bloque

cicloHacer

: 'hacer' ':' NEWLINE bloque 'mientras' '(' expresion ')' ';'? N

acceso

: IDENTIFICADOR sufijoAcceso*

sufijoAcceso

: '[' expresion ']'

| '.' IDENTIFICADOR

expresion

: '(' expresion ')'

| literal

| leer

| llamadaFuncion

| acceso

| ('!' | '-') expresion

| expresion ('*' | '/' | '%') expresion

| expresion ('+' | '-') expresion

| expresion ('<' | '>' | '<=' | '>=') expresion

| expresion ('==' | '!=') expresion

| expresion '&&' expresion

| expresion '||' expresion

literal

: ENTERO

| DECIMAL

| TEXTO

| CHARACTER

ZETARIANO

compilacion

: declaracionClase EOF

: 'int'

| 'double'

| 'char'

declaracionClase

| 'boolean'

: modificador* 'class' IDENTIFICADOR ('extends' IDENTIFICADOR)? '{'
miembro*

| 'String'

'}'

| IDENTIFICADOR

modificador

: 'public'

declaradorVariable

| 'private'

: IDENTIFICADOR ('=' inicializadorVariable)?

| 'protected'

| 'static'

inicializadorVariable

: expresion

miembro

| inicializadorArreglo

: declaracionAtributo

| declaracionConstructor

inicializadorArreglo

| declaracionMetodo

: '{' (inicializadorVariable (',' inicializadorVariable)* ','?)?

declaracionAtributo

bloque

: modificador* tipo declaradorVariable (',' declaradorVariable)* ';' : '{'
sentencia* '}'

declaracionConstructor

: modificador* IDENTIFICADOR '(' parametros? ')' bloque

sentencia

: bloque

| declaracionLocal ';'

declaracionMetodo

| 'if' '(' expresion ')' sentencia ('else' sentencia)?

: modificador* tipoRetorno IDENTIFICADOR '(' parametros? ')' bloque

| 'switch' '(' expresion ')' '{' seccionSwitch* '}'

| 'for' '(' inicioFor? ';' expresion? ';' listaExpresiones? ')'

tipoRetorno

| 'while' '(' expresion ')' sentencia

: tipo

| 'do' sentencia 'while' '(' expresion ')' ';'

| 'void'

| 'break' ';'

| 'continue' ';'

parametros

| 'return' expresion? ';'

: parametro (',' parametro)*

| 'println' '(' expresion? ')' ';'

| 'print' '(' expresion ')' ';'

parametro

| expresion ';'

: tipo IDENTIFICADOR

| ';'

tipo

: tipoBase ('[' ']')*declaracionLocal

: tipo declaradorVariable (',' declaradorVariable)*

tipoBaseseccionSwitch

: etiquetaSwitch+ sentencia*

etiquetaSwitch

: 'case' expresion ':'

| 'default' ':'

inicioFor

: declaracionLocal

| listaExpresiones

listaExpresiones

: expresion (',' expresion)*

expresion

: primario

| expresion '.' IDENTIFICADOR '(' argumentos? ')'

| expresion '.' IDENTIFICADOR

| expresion '[' expresion ']'

| expresion ('++' | '--')

| ('++' | '--' | '+' | '-' | '!') expresion

| expresion ('*' | '/' | '%') expresion

| expresion ('+' | '-') expresion

- | expression ('<' | '>' | '<=' | '>=') expression
- | expression ('==' | '!=') expression
- | expression '&&' expression
- | expression '||' expression
- | <right> expression '?' expression ':' expression
- | <right> expression ('=' | '+=' | '-=' | '*=' | '/=' | '%=') exp

primario

: '(' expression ')'

- | literal

- | 'this'

- | IDENTIFICADOR '(' argumentos? ')'

- | IDENTIFICADOR

- | 'new' IDENTIFICADOR '(' argumentos? ')'

- | 'new' tipoBase ('[' expression '])+ ('[' ''])*

- | 'new' tipoBase ('[' ''])+ inicializadorArreglo

- | 'readln' '(' '')

argumentos

: expresion (',' expresion)*

literal

: ENTERO

- | DECIMAL

- | TEXTO

- | CHARACTER

- | 'true'

- | 'false'

| 'null'

ANALIZADOR SEMÁNTICO

El análisis semántico se hace sobre los AST.

1. Construcción de tablas.

2. Validación: cada nodo del AST tiene un método analizar que valida su parte, pide el tipo de sus hijos y devuelve su propio tipo. ContextoSemantico guarda las tablas, los errores del archivo, si se está dentro de un ciclo o switch, el tipo de retorno de la función actual y la clase actual.

Validaciones que se aplican:

Regla	Cómo se valida
Imports válidos	El archivo debe existir, tener extensión .y o .z y no estar repetido.
Nombre de clase = nombre de archivo	La clase de un .z debe llamarse igual que el archivo.
Nombres repetidos	Variables en el mismo ámbito, atributos, funciones, métodos y constructores con la misma firma, estructuras/clases con el mismo nombre.
Tipos definidos	Todo tipo usado (estructura o clase) debe existir en la tabla de tipos.
Declarado antes de usarse	La variable se busca desde el ámbito actual hacia afuera sin salir del marco y su declaración debe estar antes del uso.
Compatibilidad de tipos	Asignaciones, valores iniciales, operaciones, argumentos y retornos se validan con la tabla de compatibilidad.
Condiciones	si, dum, per, if, while, for, do y el ternario deben tener una condición booleana.
Arreglos	Los índices y tamaños deben ser enteros; solo se indexa lo que es arreglo.
Listas {}	Inician arreglos o estructuras.
Llamadas	La función, método o constructor debe existir, cantidad y tipos de argumentos correctos.

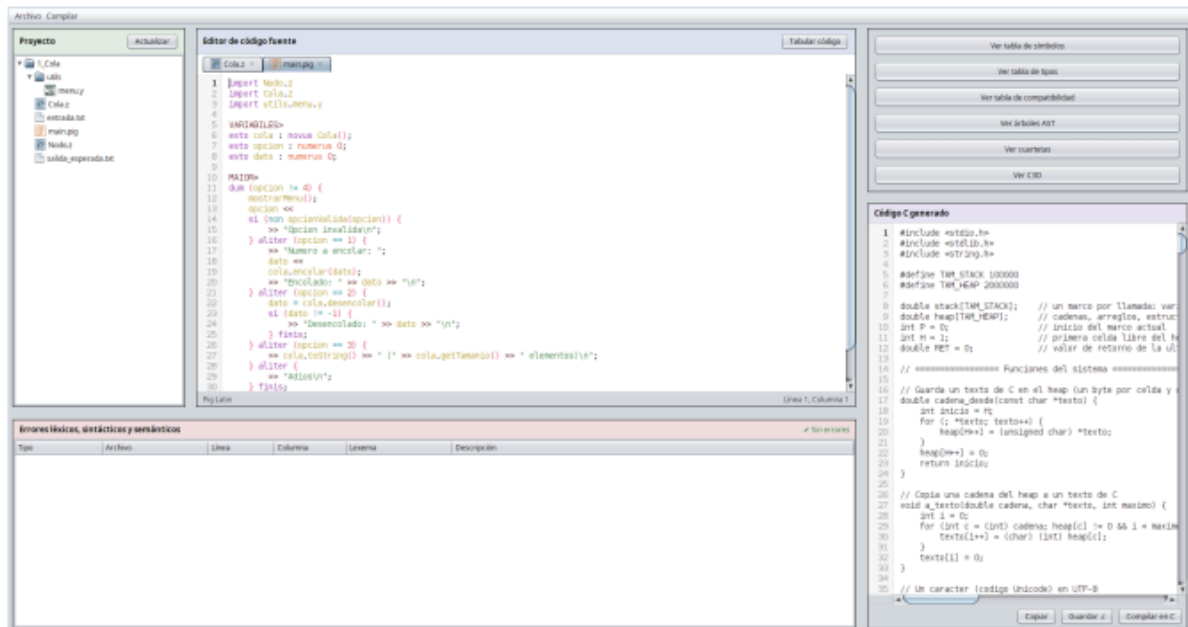
Regla	Cómo se valida
Imports válidos	El archivo debe existir, tener extensión .y o .z y no estar repetido.
Nombre de clase = nombre de archivo	La clase de un .z debe llamarse igual que el archivo.
Objetos	novus / new solo de clases; los atributos y métodos deben existir.
Retorno	return con valor en funciones con tipo y sin valor en las que no tienen tipo.
break/continue	perge, interrumpe, romper, continuar, break y continue solo dentro de ciclos.
Asignación en Zetariano	Solo a una variable, un atributo o una posición de arreglo.

TABLA DE COLOREADO

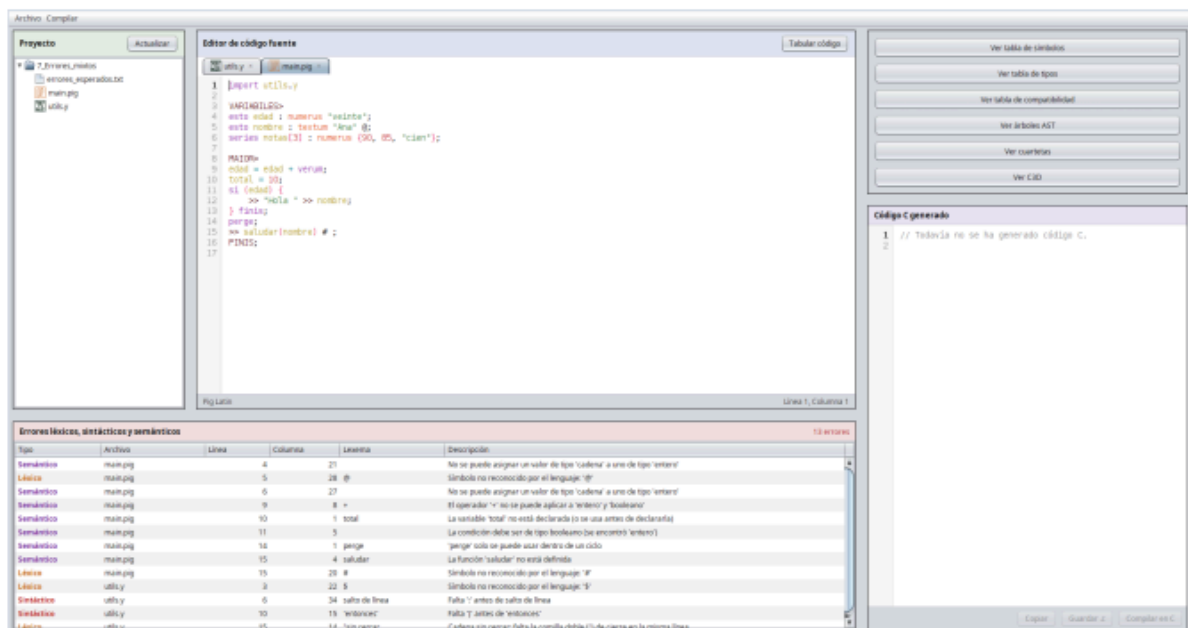
Categoría	Color
Palabras reservadas	#8E24AA
Tipos de dato	#E65100
Operadores	#00838F
Identificadores	#B39800
Números	#C62828
Cadenas y caracteres	#2E7D32
Signos de agrupación	#E91E63
Secciones y funciones especiales	#590202
Comentarios	#8A8A8A
Todo lo demás	#000000

INTERFAZ GRÁFICA

Se propuso este diseño amigable con el usuario.



Proyecto compilado sin errores: el panel derecho muestra el código C generado



Proyecto con errores léxicos, sintácticos y semánticos en la tabla de errores